TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT PROGRESS REPORT ON DQN-BASED TRAFFIC SIGNAL CONTROL FOR ROAD TRAFFIC FLOW IMPROVEMENT AT THE THAPATHALI INTERSECTION

Submitted By:

Hemanta Bhusal	(THA079BCT013)
Pratyoush Silwal	(THA079BCT030)
Sachin Gautam	(THA079BCT035)
Shoviyat Chapagain	(THA079BCT041)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

December 2025

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A MINOR PROJECT PROGRESS REPORT ON DQN-BASED TRAFFIC SIGNAL CONTROL FOR ROAD TRAFFIC FLOW IMPROVEMENT AT THE THAPATHALI INTERSECTION

Submitted By:

Hemanta Bhusal	(THA079BCT013)
Pratyoush Silwal	(THA079BCT030)
Sachin Gautam	(THA079BCT035)
Shoviyat Chapagain	(THA079BCT041)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

In partial fulfillment for the award of the

Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Saroj Sakhya

December 2025

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing the academic environment and resources necessary to initiate this project. We extend our heartfelt appreciation to our project supervisor, **Er. Saroj Sakhya** , for his invaluable guidance, consistent support, and constructive feedback throughout this stage.

We are also grateful to the traffic police personnel and concerned authorities for sharing practical insights regarding the existing traffic signal implementation and on-ground traffic management practices at the project location. Their input helped ensure that our proposed work remains aligned with real-world operational conditions. Finally, we would like to thank our teachers, classmates, and friends for their continuous encouragement and support.

Hemanta Bhusal	(THA079BCT013)
Pratyoush Silwal	(THA079BCT030)
Sachin Gautam	(THA079BCT035)
Shoviyat Chapagain	(THA079BCT041)

ABSTRACT

Traffic congestion at the Thapathali intersection is a significant issue, causing delays, inefficiencies, higher environmental impacts, and increased travel stress for commuters. Manual traffic control, primarily handled by traffic personnel, relies on the fixed-time signal method and personal judgment, often without considering real-time congestion levels, leading to suboptimal performance under varying traffic conditions. This project proposes an intelligent traffic control system that uses Deep Reinforcement Learning (DRL) to optimize traffic flow at the intersection. A DRL agent based on the Deep Q-Network (DQN) algorithm is developed and trained in a Simulation of Urban MObility (SUMO)-simulated environment to learn optimal traffic signal timing policies. The system receives input data such as the Origin-Destination (OD) vehicle count, while other traffic metrics like average waiting times, throughput, and average travel time are calculated within the Reinforcement Learning (RL) framework to inform decision-making for controlling the traffic signal. The system's performance is compared with the fixed-time control method, showing improvements in reducing traffic delays, improving throughput, and optimizing overall intersection performance. The proposed system is designed to reduce queue lengths and throughput enhancing traffic flow efficiency.

Keywords: *Adaptive Control, DQN, DRL, Max-Pressure Theory, Queue Length, SUMO, Throughput, Traffic Signal Control*

TABLE OF CONTENTS

ACKNOWLEDGEMENT . . . . . **i**

ABSTRACT . . . . . **ii**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vii**

LIST OF ALGORITHMS . . . . . **viii**

LIST OF ABBREVIATIONS . . . . . **ix**

LIST OF SYMBOLS . . . . . **x**

1. Introduction **1**
- 1.1. Background 1
- 1.2. Motivation 1
- 1.3. Problem Statement 2
- 1.4. Objectives 2
- 1.5. Scope and Limitations 2

2. Literature Review **4**
- 2.1. Conventional Traffic Signal Control Methods 4
- 2.2. RL for Traffic Signal Control 4
- 2.3. DRL and Value-Based Control with DQN 5
- 2.4. Network-Level Coordination as Context for Future Extension 5
- 2.5. Structured Policy Learning via Phase Competition and Symmetry . . . 6
- 2.6. Simulation and Realistic Data Grounding 6
- 2.7. Summary and Project Positioning 6

3. Requirements Analysis **8**
- 3.1. Functional Requirements 8
- 3.2. Non-Functional Requirements 8
- 3.3. System Requirements 9

4. System Architecture and Methodology **10**
- 4.1. System Architecture 10
- 4.2. Data Analysis 10
- 4.3. Environment Setup for Traffic Simulation 12
- 4.4. DQN Agent Development 14

4.5. Training Process 15
4.6. Algorithms Used 16

5. Implementation Details 17
5.1. Vehicle Flow Generation 17
5.2. Traffic Simulation Environment (SUMO) 20
5.3. Environment Wrapper Implementation 22
5.4. Traffic Flow at Thapathali Intersection 22
5.5. DQN Agent Implementation 27
5.6. Training Pipeline Implementation 27
5.7. Configuration Management 27

6. Results and Analysis 28
6.1. Simulation Environment Configuration 28
6.2. Performance Comparison Results 28
6.3. Reward Analysis and Learning Trajectory 29
6.4. Cumulative Performance Evaluation 29
6.5. Statistical Performance Distribution 30
6.6. Queue Management Effectiveness 31
6.7. Reward Distribution Patterns 32
6.8. Conclusion and Practical Implications 33

7. Remaining Tasks 35
7.1. Completed Tasks 35
7.2. Remaining Milestones 35
7.3. Blockers and Mitigations 35
7.4. Future Work 36

8. Conclusion 37
8.1. Achievements 37
8.2. Limitations 37
8.3. Future Work 37
8.4. Final Remarks 37

APPENDICES 38

A. Code Snippets 38
A.1. DQN Network Architecture 38
A.2. Replay Buffer 38
A.3. DQN Agent 39
A.4. Model Training 39

A.5. Model Saving and Loading 40
A.6. Summary 40

B. Project Budget 41

C. Project Timeline 42
C.1. Gantt Chart 42

D. Summary of Data 43

REFERENCES 46

LIST OF FIGURES

Figure 4-1. Block Diagram of DQN-Based Traffic Signal Control System 10
Figure 4-2. SUMO Environment Block Diagram 14
Figure 4-3. Neural Network Architecture for Q-value Prediction 15
Figure 5-1. Thapathali Road Traffic flow and Traffic light control stage representation 23
Figure 5-2. Implemented Traffic Signal Phases for fixed-timer and DQN-based Agent 25
Figure 6-1. SUMO Simulation Environment of Thapathali Intersection 28
Figure 6-2. Step-wise Reward Comparison: DQN vs Fixed-time Control 29
Figure 6-3. Cumulative Reward Performance Over Time 30
Figure 6-4. Statistical Comparison of Performance Metrics 31
Figure 6-5. Queue Length Comparison: DQN vs Fixed-time Control 32
Figure 6-6. Reward Distribution Patterns of Both Control Methods 33
Figure C-1. Project Gantt Chart 42

LIST OF TABLES

Table 2.7.1. Comparison of Traffic Signal Control Methods 7
Table 3.1.1. Functional Requirements for the Project 8
Table 3.2.1. Non-Functional Requirements for the Project 8
Table 4.2.1. Vehicle Class and Adopted PCU Factors 11
Table 5.2.1. Vehicle Type Configurations in SUMO Simulation 21
Table B.0.1. Project Budget Allocation 41
Table D.0.1. 15-Minute Interval Vehicle Counts by Class: Day 1 (06:00–15:15) . 43
Table D.0.2. 15-Minute Interval Vehicle Counts by Class: Day 1 (15:15–22:15) . 44
Table D.0.3. PCU OD Data: Days 1 & 2 44
Table D.0.4. PCU OD Data: Days 3 & 4 45
Table D.0.5. PCU OD Data: Day 5 45

LIST OF ALGORITHMS

4.6.1. Max-Pressure Reward Calculation 16
4.6.2. DQN Training Process 16
5.1.1. Perfect Vehicle Allocation with Remainder Tracking 17
5.1.2. Batch Perfect Allocation (No Time Interpolation) 18
5.1.3. Mathematical Formulation 19

LIST OF ABBREVIATIONS

API	Application Programming Interface
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ML	Machine Learning
MSE	Mean Squared Error
OD	Origin-Destination
OSM	OpenStreetMap
PCU	Passenger Car Unit
PEP8	Python Enhancement Proposal 8
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
SUMO	Simulation of Urban MObility
TRACI	Traffic Control Interface
XML	eXtensible Markup Language

LIST OF SYMBOLS

α	Learning rate or significance level
$\approx$	Approximately equal to
β	Decay factor or regularization parameter
Δ	Difference or change operator
ϵ	Error term or small positive value
$\equiv$	Equivalent to
γ	Discount factor
λ	Eigenvalue or wavelength
μ	Mean or average value
∇	Gradient operator
Ω	Sample space or angular frequency
∂	Partial derivative operator
π	Policy function or mathematical constant
$\prod$	Product
σ	Standard deviation or summation operator
$\subset$	Subset of operator
$\sum$	Summation
θ	Parameter vector or angle

1. INTRODUCTION

1.1. Background

In urban cities like Kathmandu, traffic congestion is a growing problem, causing wasted time, pollution, and economic loss. Currently, in Kathmandu, traffic is monitored through cameras, and signal timings are set manually based on these observations or managed manually by traffic police. However, these methods are not very effective; instead of reducing congestion, they often cause additional delays because this system often fails during sudden surges, accidents, or unusual traffic patterns. This limitation highlights the need for more intelligent, adaptive traffic management systems. Since most delays originate at major junctions, this project focuses specifically on improving intersection-level signal control at the Thapathali intersection rather than addressing city-wide traffic management.

1.2. Motivation

The daily commute in Kathmandu, particularly around busy intersections like Thapathali, has long been a source of frustration for its residents. Traffic congestion is a persistent issue, especially as the city continues to grow and traffic patterns become increasingly unpredictable. Conventional traffic control methods, which rely on fixed patterns and limited flexibility, are proving insufficient to address the dynamic and often chaotic nature of Kathmandu's roadways. As the city's population expands and the number of vehicles on the road increases, the need for a more intelligent and adaptive approach to traffic signal control becomes critical.

This project seeks to tackle the limitations of traditional traffic management systems by employing the power of DRL. The aim is to develop an adaptive traffic signal control framework that can dynamically respond to real-time traffic conditions, improving traffic efficiency, reducing congestion, and making daily commutes smoother for the people of Kathmandu, especially around congested areas like Thapathali.

To test and validate this approach, we turn to advanced simulation tools like SUMO, which offers a scalable and safe platform for replicating real-world traffic scenarios. With accurate road network data from OpenStreetMap (OSM), SUMO allows us to simulate the conditions in Kathmandu and evaluate AI-based traffic control strategies. This simulation environment provides an ideal space to refine the system before real-world implementation.

1.3. Problem Statement

Current Thapathali intersection traffic signal systems are largely static or semi-adaptive, failing to respond effectively to complex and fluctuating traffic patterns. This leads to increased vehicle waiting times and travel delays, higher fuel consumption and air pollution, and inefficient coordination across intersections. The problem this project addresses is how to design a traffic signal control system that can adapt in real-time to dynamic traffic conditions, using a learning-based approach to optimize flow, minimize delays, and improve overall intersection throughput.

A DQN-based traffic signal controller is expected to reduce average waiting time and queue length compared to fixed-time control under realistic Thapathali traffic demand because it can continuously adjust phase selection and green allocation based on real-time traffic states (e.g., queue lengths, waiting times, current phase, and elapsed duration). Unlike fixed-time plans that rely on pre-set splits and cannot respond to demand fluctuations, asymmetric approach volumes, or sudden surges, the DQN agent can reduce wasted green time on low-demand approaches, prevent queue buildup and spillback, and improve throughput under the same demand conditions.

1.4. Objectives

- To develop a DQN-based DRL agent for adaptive traffic signal control using SUMO in a realistic simulation of the Thapathali Intersection.
- To compare the performance of the proposed agent against conventional traffic signal control methods.

1.5. Scope and Limitations

This project focuses on developing a DQN-based DRL agent for adaptive traffic signal control using realistic simulation of the Thapathali intersection. The scope defines what will be covered in the project and sets clear boundaries for its implementation. The intended applications include simulation-based testing of signal timing strategies for a Thapathali intersection, comparative performance analysis against baseline signal plans, and decision support for traffic engineering studies prior to real-world deployment. The proposed system is designed to be scalable; subject to time and data availability, it can be extended to multi-intersection coordination in future work.

1.5.1. Scope

- To configure the Thapathali intersection map into SUMO to simulate realistic traffic patterns.

- To design the DQN-based DRL agent with state representations, reward functions, and action spaces.
- To evaluate model performance using metrics such as average waiting time, queue lengths, and vehicle throughput.

1.5.2. Limitations

- No hardware implementation or real-world field testing with traffic signals.
- Pedestrians and public transport are excluded in this phase to keep the control problem tractable and focus on vehicle-delay reduction at Thapathali; modeling them requires additional constraints and reliable data.
- Extreme scenarios or multi-modal integration, like ride-sharing or parking systems, are not considered.

2. LITERATURE REVIEW

Urban intersections are recurring bottlenecks where conflicting traffic streams compete for limited green time. Conventional signal timing can improve intersection performance, but it often depends on simplified assumptions and periodic manual retiming. In practice, traffic demand varies by time of day and day type, and static plans may not remain effective under changing conditions. For the Thapathali Intersection specifically, prior local work has evaluated the intersection as an isolated study area using observed peak-hour traffic volumes and operational performance measures (e.g., delay, queue-related indicators, and level of service), providing realistic context and baseline conditions for simulation-based signal studies [1].

2.1. Conventional Traffic Signal Control Methods

Traditional signal control approaches for an isolated intersection commonly produce a cycle-based plan consisting of a phase sequence, cycle length, and green splits. A widely used classical method is Webster-type timing, which computes timing parameters from traffic volume and related assumptions to generate a fixed-time plan for an individual intersection [2]. Beyond fixed-time plans, actuated and rule-based controllers adjust phase changes based on detector inputs and predefined rules, offering limited responsiveness without learning [2]. At a higher level, transportation methods can be grouped into (i) fixed-time control, (ii) actuated/rule-based control, (iii) selection-based adaptive control (choosing among precomputed plans), and (iv) optimization-based adaptive control that explicitly optimizes an objective under simplified flow models [2]. These approaches are practical and interpretable, but they can require careful calibration, may rely on oversimplified traffic representations, and may not adapt effectively to complex, time-varying patterns.

A particularly relevant conventional baseline is max-pressure control, which uses real-time queue information to select phases that balance upstream and downstream congestion [2]. Max-pressure has theoretical properties related to throughput under certain conditions, making it a strong comparator for learning-based controllers. However, it remains a greedy decision rule and does not learn long-term policies from interaction.

2.2. RL for Traffic Signal Control

RL reframes signal control as a sequential decision-making process: an agent observes traffic conditions, takes an action (e.g., choosing the next phase or adjusting green time), and receives a reward reflecting operational quality (e.g., reduced delay or queues). RL is attractive for traffic signal control because “optimal” timing is difficult to label

in advance for all scenarios; instead, policies can be learned through trial-and-error interaction in a simulator [2]. For a single intersection, the main design choices typically involve: (1) the state representation (what the agent observes), (2) the action space (what the agent can control), and (3) the reward function (what the agent is incentivized to optimize). Practical state elements often include per-lane vehicle counts/queues and the current signal phase, while rewards are usually derived from delay/queue metrics or throughput-related measures [2, 3].

2.3. DRL and Value-Based Control with DQN

DRL extends RL by using neural networks to approximate value functions or policies, enabling learning from richer traffic states. A common value-based method is the DQN, which estimates action-values and can be trained using experience replay to stabilize learning [4]. A key challenge in many DRL traffic-signal studies is that performance can be highly sensitive to reward and state design; overly complex states and heuristic rewards can slow training or lead to unstable outcomes. To address this, one approach explicitly grounds DRL design in transportation theory by connecting RL to max-pressure concepts: the reward is shaped to align with pressure-based objectives, and the state is designed to retain only the information necessary to optimize that objective [4]. This is important even for a single-intersection project because principled reward/state choices can improve learning stability and reduce unnecessary modeling complexity.

In addition to reward/state design, evaluation practice in DRL traffic signal control typically compares learned policies to conventional baselines such as fixed-time plans and other transportation-inspired controllers, using metrics like travel time, delay, queue length, and throughput to quantify benefits [2, 4]. For single-intersection studies, these comparisons are crucial to demonstrate that learning provides value beyond strong non-learning baselines.

2.4. Network-Level Coordination as Context for Future Extension

Although the present project targets one intersection, the literature on multi-intersection control highlights why coordination is beneficial and how learning methods represent neighboring influence. One DRL approach models traffic signal control as a Markov game where each intersection is controlled by an agent, and neighboring intersections influence one another through learned cooperation [3]. In that formulation, an agent's observation includes its current phase (often as a one-hot vector) and lane-level vehicle counts, while a learned attention mechanism assigns different weights to neighbors depending on spatial and temporal conditions [3]. The method's Q-learning style objective reflects value-based learning with a function approximator and a loss that penalizes Q-value prediction errors across intersections and time steps [3]. Even when

applied to a single intersection, these ideas motivate using lane-level counts and explicit phase information as core state variables and reinforce the importance of scalable representations if later extension to multi-intersection coordination is desired.

2.5. Structured Policy Learning via Phase Competition and Symmetry

Another relevant direction improves learning efficiency by structuring how actions (phases) are represented. A phase-competition approach argues that many traffic states are equivalent under rotation and flipping (symmetry) and that RL agents should generalize across such cases without relearning from scratch [5]. The method leverages this observation by modeling competition among phases, prioritizing movements with higher demand when conflicts exist, and explicitly designing the model to be invariant to symmetric transformations [5]. For a single intersection, this supports two practical design lessons: (1) the action space should reflect meaningful phase choices rather than overly granular controls that increase exploration burden, and (2) the model/state design can exploit structural regularities in intersection layouts to improve sample efficiency.

2.6. Simulation and Realistic Data Grounding

DRL traffic signal control is commonly developed and validated in simulation for safety, repeatability, and fast iteration. SUMO is frequently used as it can execute signal actions, simulate vehicle movements, and provide state measurements required by controllers [5]. For Thapathali, local operational data provides a valuable foundation for building realistic simulation demand and for defining baseline performance expectations. The Thapathali case study reports peak-hour traffic volumes (in Passenger Car Unit (PCU)) for multiple observation days and provides approach-level volume distributions, which can be mapped into simulation flows [1]. It also reports performance statistics such as degree of saturation, average delay, level of service, and queue-related measures for the existing intersection operation and for candidate improvement options [1]. Such data is particularly useful for a minor project because it enables (i) calibration of traffic demand patterns in simulation, and (ii) evaluation of whether learned signal policies improve meaningful operational indicators under realistic conditions.

2.7. Summary and Project Positioning

Overall, the literature indicates a progression from fixed-time and rule-based controllers toward learning-based adaptive control capable of reacting to dynamic traffic states. Conventional transportation methods remain important as baselines and as sources of theory-driven guidance (e.g., max-pressure). DRL methods such as DQN provide a practical foundation for adaptive control, but their success depends strongly on state, action, and reward design. Theory-grounded DRL designs linked to established

traffic principles can improve stability and interpretability [4], while attention-based cooperation and symmetry-aware phase modeling illustrate pathways for scalability and efficiency [3, 5]. For a single-intersection implementation at Thapathali, a DQN-based DRL agent trained in SUMO and grounded in realistic local traffic volumes provides a manageable scope while still aligning with the most effective practices observed in the literature [1, 2].

Table 2.7.1. Comparison of Traffic Signal Control Methods

Method	**Control Logic**	**Adaptivity**	**Data Needs**
Fixed Time	Pre-set cycle length, phase order, and green splits (static plan).	Low: no real-time response; only changes if manually retimed.	Historical/observed volumes for timing design; no training required.
Max Pressure	Selects phase with highest pressure using upstream vs downstream queue imbalance (rule-based).	High: reacts instantly to queues, but remains a fixed greedy rule (no learning).	Real-time queue/occupancy (ideally downstream too); no training dataset.
DQN-based DRL	Learns a signal policy by maximizing reward through simulation interaction.	High: learns and adapts a policy to objectives and constraints.	Training episodes in SUMO; state features; compute and tuning effort.

3. REQUIREMENTS ANALYSIS

3.1. Functional Requirements

Table 3.1.1. Functional Requirements for the Project

ID	Description
FR-001	The system shall import the Thapathali intersection map from OpenStreetMap and configure it in SUMO with realistic traffic patterns.
FR-002	The system shall implement a DQN-based agent capable of changing traffic signal phases to optimize traffic flow.
FR-003	The system shall update the DQN target network periodically to facilitate stable real-time learning.
FR-004	The system shall accept traffic state inputs including vehicle counts, queue lengths, and waiting times.
FR-005	The system shall output traffic light control commands via Traffic Control Interface (TRACI) based on the DQN agent's decisions.
FR-006	The system shall log key performance metrics (e.g., queue length, throughput) for post-simulation analysis.
FR-007	The system shall visualize simulation progress via SUMO-GUI for monitoring purposes.
FR-008	The system shall allow comparative evaluation against baseline fixed-time or actuated signal plans.

3.2. Non-Functional Requirements

Table 3.2.1. Non-Functional Requirements for the Project

ID	Category	Description
NFR-001	Performance	The DQN agent shall compute and apply a new traffic signal phase within 1 second per simulation step.
NFR-002	Performance	The system shall achieve at least **15–20% reduction in average waiting time** compared to the baseline signal plan.
NFR-003	Performance	The system shall achieve at least **10% increase in vehicle throughput** compared to the baseline.
NFR-004	Portability	The system shall be fully operational on Windows 10+ and Ubuntu 20.04+.
NFR-005	Usability	The system shall provide real-time visualization options (SUMO-GUI) and clear performance logs.
NFR-006	Maintainability	The code shall be modular, well-documented, and follow Python Enhancement Proposal 8 (PEP8) conventions.
NFR-007	Security	No personal or sensitive data is processed; only aggregated traffic metrics are used.

3.3. System Requirements

3.3.1. Hardware Requirements

- **Minimum:** 64-bit CPU, 8 GB RAM, 10 GB free disk space.
- **Recommended:** 4+ core CPU, 16 GB RAM, SSD storage.
- **Optional (Training):** Dedicated GPU with 4+ GB VRAM or Google Colab access.

3.3.2. Software Requirements

- **Operating System:** Windows 10+ or Ubuntu 20.04+.
- **SUMO:** Eclipse SUMO 1.26.0 or above.
- **Python:** 3.8 or higher.
- **Key Libraries:** TRACI, pandas, PyTorch, NumPy, Matplotlib.
- **Development Tools:** VS Code, Git, and GitHub.

4. SYSTEM ARCHITECTURE AND METHODOLOGY

4.1. System Architecture

The system employs a DQN architecture for traffic signal optimization, integrating SUMO as the simulation environment with a custom reinforcement learning framework. The state representation module processes real-time traffic data including lane vehicle counts and current phase status into a 22-dimensional feature vector (6 incoming lanes, 12 outgoing lanes, and 4 phase indicators). This vector feeds into dual neural networks: a primary network that updates continuously via backpropagation and a target network that stabilizes training through periodic weight synchronization. The networks feature fully connected hidden layers with 128 and 64 neurons designed to capture complex non-linear relationships in traffic states. A replay buffer stores 10,000 experience tuples to enable batch learning and break temporal correlations, while an ϵ-greedy policy balances exploration and exploitation. The action space comprises four discrete traffic signal controls, with rewards calculated based on the Max-Pressure metric (pressure difference between incoming and outgoing lanes). This modular design ensures stable learning while maintaining real-time responsiveness to dynamic traffic conditions.

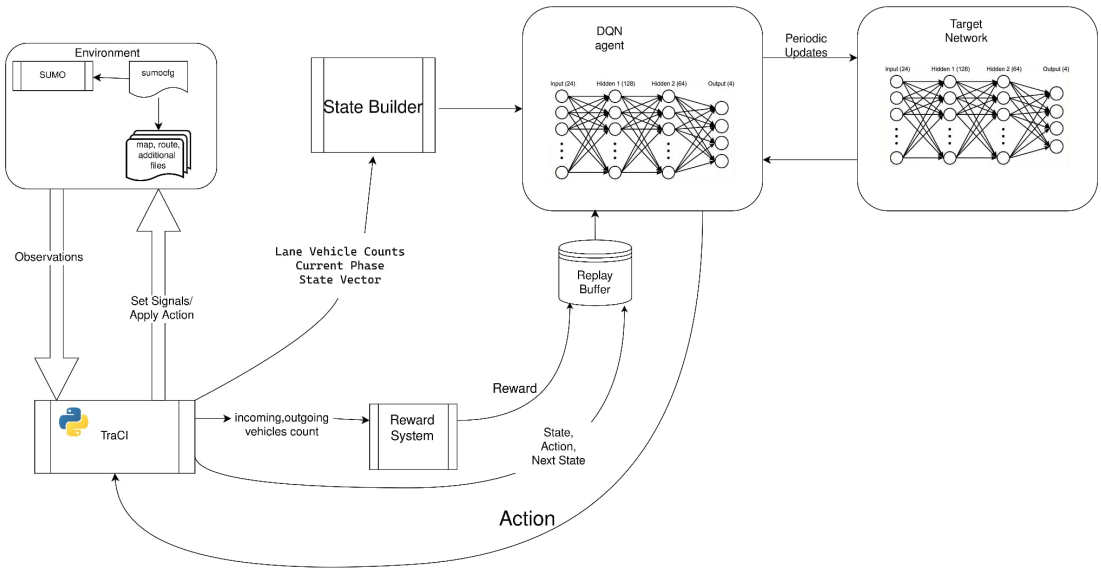

Figure 4-1. Block Diagram of DQN-Based Traffic Signal Control System

4.2. Data Analysis

4.2.1. Data Collection

For the traffic network modeling, primary traffic flow data was sourced from the research paper *A Case Study of Thapathali Intersection* by Lav Maharjan [1]. The data collection methodology involved comprehensive field surveys over a duration of

five days, strategically selected to capture both weekday (3 days) and weekend (2 days) traffic characteristics. The observations were conducted continuously from 6:00 AM to 10:15 PM, ensuring the inclusion of morning and evening peak periods as well as off-peak transitions.

The data collected during the surveys were represented in terms of PCU OD data and vehicle counts for different time intervals. PCU OD data was recorded for two specific time intervals: one during the morning peak period and one during the evening peak period, each lasting one hour. These intervals were selected to capture the highest traffic flow during the day. The vehicle counts were recorded separately for these intervals within the given time frame. The representation of the data allows for a clearer analysis of traffic behavior during peak and off-peak times, providing a foundation for modeling the traffic flow more accurately.

4.2.2. Data Representation

Table 4.2.1. Vehicle Class and Adopted PCU Factors

Vehicle Class	Representation	Adopted PCU Factor
Light Vehicle	LV	1.5
Heavy Vehicle	HV	3
Bus	B	3
Mini Bus	MB	2.5
Micro Bus	MIB	1.5
Car	C	1
Four-Wheel	F	1
Tempo	T	0.75
Utility	U	0.75
Two-Wheeler	TW	0.25

4.2.3. OD Data Allocation

The vehicle allocation system is implemented using a Python script, leveraging libraries like `pandas`, `numpy`, and `matplotlib` for data manipulation and analysis. The following summarizes the key components of the implementation.

The primary input data consists of:

- **Vehicle Count Data (`vehicle_count.xlsx`):** Contains the number of vehicles for various vehicle classes during 15-minute intervals.
- **PCU OD Data (`pcu_od.xlsx`):** Provides the proportions of PCU allocated to each OD pair for AM and PM periods.

Data Cleaning

- Non-numeric characters are removed from vehicle count and PCU data.
- Invalid data is replaced with zero.
- Data is converted to numeric format for proper calculation and allocation.
- Vehicles with low counts were grouped into similar vehicle types.
- Vehicles with same PCU and sizes were grouped into one vehicle type.

Vehicle Allocation Process

For each vehicle class and OD pair, the following process is followed:

- **Proportions Calculation:** AM and PM proportions are calculated based on the PCU data.
- **Allocation via Largest Remainder Method:** Vehicles are initially allocated proportionally, and leftover vehicles are distributed to ODs with the largest fractional values.
- **Piecewise Interpolation:** For each 15-minute interval, vehicle proportions are dynamically interpolated using piecewise interpolation.

Maternity Restrictions

For certain ODs related to maternity, only specific vehicle classes are allowed (e.g., cars, two-wheelers, four-wheel vehicles, and utility vehicles), based on real-world traffic constraints.

Output Generation

The final allocated vehicle counts are saved to Excel files, including:

- 15-minute interval counts for every OD and vehicle class.
- A summary sheet showing total vehicles and AM/PM proportions.
- A verification sheet ensuring correct and consistent allocation.

4.3. Environment Setup for Traffic Simulation

The first step in the environment setup for traffic simulation using SUMO was to use the NETEDIT tool to create a road network that accurately represents the Thapathali intersection. This tool allows for the design of road networks, including multiple lanes, junctions, and traffic signals. The geometry of the intersection and road structure was designed in NETEDIT and saved as the `map.net.xml` file. This file contains the road network's topography, including nodes (junctions), edges (roads), and their connections.

It also defines traffic signals at each intersection, which are crucial for simulating realistic traffic flow.

Once the network structure was defined, vehicle movement was modeled using the `routes.rou.xml` file. This file specifies the routes that vehicles will take through the network and includes information such as departure times, vehicle types, and flow. The vehicle flow is generated to ensure that vehicles enter the network at regular intervals, with a preference for the least congested lane, thus simulating realistic driving behavior.

The `vtypes.xml` file defines various vehicle types used in the simulation, each with specific attributes such as length, maximum speed, acceleration, and deceleration. These characteristics are essential for accurately simulating vehicle behavior across different classes.

The `flows.xml` file captures vehicle flow data and traffic signal phases. This file defines how many vehicles pass through each edge of the network in specific time intervals and the corresponding traffic signal phases.

To interact with the simulation in real-time, the TRACI interface is used. This allows external scripts to control the simulation, modifying vehicle flows and traffic signal phases dynamically. A Python script adjusts the signal phases based on real-time data, such as vehicle counts and congestion levels.

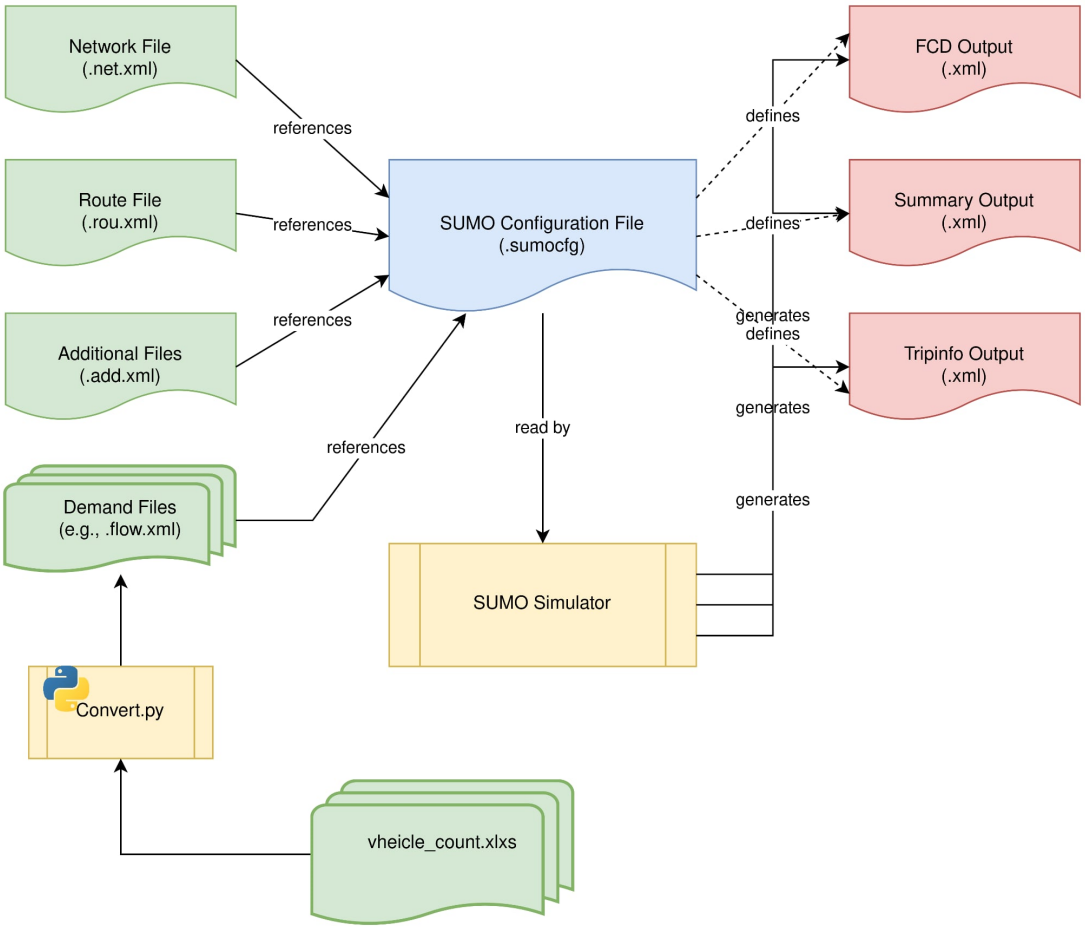

Figure 4-2. SUMO Environment Block Diagram

4.4. DQN Agent Development

4.4.1. State Representation

The DQN agent observed key traffic conditions, including vehicle counts on each lane, queue lengths, current traffic signal phases, and vehicle waiting times. These parameters were combined into a 22-dimensional state vector:

$$s_t = [\underbrace{n_1, n_2, \ldots, n_{in}}_{\text{Incoming Counts}}, \underbrace{n_{out,1}, \ldots, n_{out,m}}_{\text{Outgoing Counts}}, \underbrace{0, 1, 0, 0}_{\text{Current Phase}}]$$

4.4.2. Action Space

The agent selects from four predefined traffic signal phases. Each phase remains green for 15 seconds, followed by a 3-second yellow transition:

- **Action 1:** Straight movements between Maitighar and Kupondole.

- **Action 2:** Right turn from Maitighar to Tripureshwor + Maitighar to Kupondole.
- **Action 3:** Straight from Tripureshwor to Maternity + Maternity left movements.
- **Action 4:** Right from Kupondole to Maternity + Maitighar straight movements.

4.4.3. Reward Function

A reward function based on Max-Pressure Theory was used. Pressure was calculated as the difference between incoming and outgoing vehicle densities:

$$R = -\sum_{i \in L_{\text{in}}} \left(w(i) - \frac{1}{|L_{\text{out}}|} \sum_{j \in L_{\text{out}}} w(j) \right) - \lambda \times (\text{max waiting time}) \tag{4-1}$$

4.4.4. Neural Network Architecture

The agent uses a deep neural network with an input layer, two hidden layers (128 and 64 neurons), and an output layer for action values.

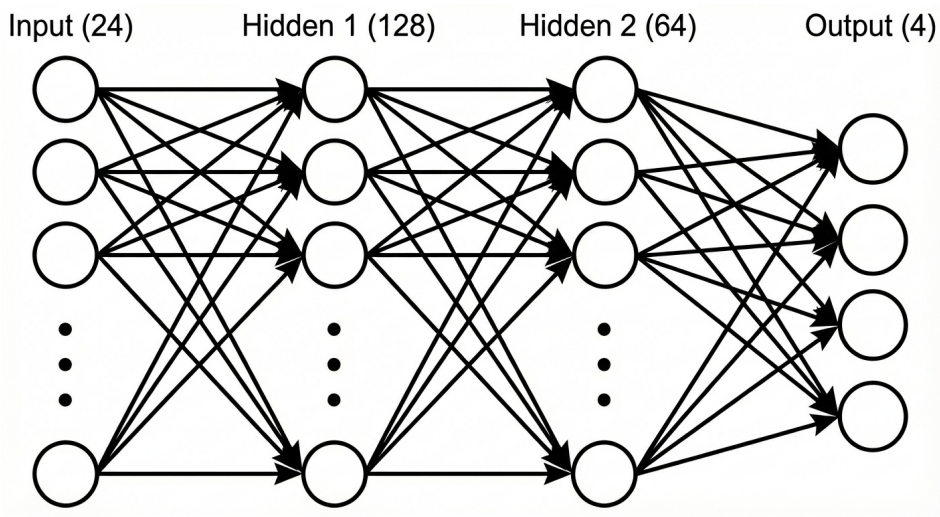

Figure 4-3. Neural Network Architecture for Q-value Prediction

4.5. Training Process

4.5.1. Learning Strategy

An ϵ-greedy strategy was used. The agent initially explored the environment ($\epsilon = 1.0$) and gradually reduced ϵ to 0.01 to favor exploitation of learned strategies.

4.5.2. Experience Replay

Experience replay improved stability by storing transitions in a 10,000-capacity buffer. Random samples were used for training to break temporal correlations.

4.5.3. Training Loop

The model was trained over 500 episodes. The process included:

1. Calculate Target Q: $Q_{\text{target}} = r + \gamma \times \max(Q(s', a'))$.
2. Calculate Current Q: $Q_{\text{current}} = Q(s, a)$.
3. Backpropagate Loss: $\text{MSE}(Q_{\text{target}}, Q_{\text{current}})$.
4. Update policy network weights.

4.6. Algorithms Used

Algorithm 4.6.1 Max-Pressure Reward Calculation

```
Input: Incoming/Outgoing lane vehicle counts
Output: Reward value
procedure CALCULATEMAXPRESSUREREWARD
    total_pressure ← 0
    for all incoming_lane ∈ incoming_lanes do
        incoming_count ← CountVehiclesWaiting(incoming_lane)
        outgoing_average ← CalculateAverageOutgoingTraffic()
        pressure ← incoming_count − outgoing_average
        total_pressure ← total_pressure + pressure
    end for
    reward ← −total_pressure − λ × (max_waiting_time)
    return reward
end procedure
```

Algorithm 4.6.2 DQN Training Process

```
procedure TRAINDQN(E, B, F)
    Initialize ϵ ← 1.0, ϵ_min ← 0.01, ϵ_decay ← 0.995
    Initialize replay buffer and networks
    for episode ← 1 to E do
        Reset environment and get state
        while simulation is running do
            action ← SelectAction(state, ϵ)
            Execute action, observe reward and next_state
            Store (state, action, reward, next_state) in buffer
            if buffer has enough samples then
                Train Q-network using random batch
            end if
            state ← next_state
        end while
        Update ϵ and periodically sync target network
    end for
end procedure
```

5. IMPLEMENTATION DETAILS

This project focuses on developing an intelligent traffic control system using DQN to optimize traffic flow at the Thapathali Intersection in Kathmandu. By applying DQN, the system dynamically adjusts traffic light phases based on real-time data such as vehicle density, waiting times, and congestion levels. The primary objective is to minimize vehicle wait times and enhance traffic throughput by learning and implementing the most efficient signal phases.

The system utilizes SUMO for simulating traffic scenarios and TRACI to enable real-time interaction between the DQN model and the traffic simulation. This integration allows the DQN model to control traffic signal phases, assess the impact of its actions, and continuously improve the traffic flow based on the simulation feedback.

5.1. Vehicle Flow Generation

Algorithm 5.1.1 Perfect Vehicle Allocation with Remainder Tracking

```
Input:
    vehicle_counts: Dictionary {vc_1 : count_1, vc_2 : count_2, ...}
    proportions: Dictionary {od_1 : p_1, od_2 : p_2, ...}
    od_names: List of OD names [od_1, od_2, ..., od_N]
    prev_remainders: Dictionary {od_1 : r_1, od_2 : r_2, ...} (Optional)
Output:
    allocation: Dictionary {od_1 : {vc_1 : count, ...}, ...}
    new_remainders: Dictionary {od_1 : r_1, od_2 : r_2, ...}
procedure ALLOCATEINTERVALPERFECT(vehicle_counts, proportions,
od_names, prev_remainders)
    if prev_remainders is None then
        Initialize prev_remainders ← {od : 0 for all od ∈ od_names}
    end if
    Initialize allocation ← {od : {} for all od ∈ od_names}
    Initialize new_remainders ← {od : 0 for all od ∈ od_names}
    for all (vehicle_class, total_count) ∈ vehicle_counts do
        if total_count = 0 then
            continue
        end if
        for all od ∈ od_names do
            exact ← total_count × proportions[od] + prev_remainders[od]
            allocated ← ⌊exact⌋
            remainder ← exact − allocated
            allocation[od][vehicle_class] ← allocated
            new_remainders[od] ← remainder
        end for
    end for
    return (allocation, new_remainders)
end procedure
```

Algorithm 5.1.2 Batch Perfect Allocation (No Time Interpolation)

```
procedure ALLOCATEVEHICLESPERFECT(vehicle_counts, proportions, od_names)
    Initialize result ← {od : {vc : 0 for all vc} for all od ∈ od_names}
    for all (vehicle_class, total_count) ∈ vehicle_counts do
        if total_count = 0 then
            continue
        end if
        targets ← {}                                        ▷ Exact target allocations
        for all od ∈ od_names do
            targets[od] ← total_count × proportions[od]
        end for
                                                            ▷ Step 1: Allocate integer parts
        allocated ← {}
        fractions ← {}
        for all od ∈ od_names do
            allocated[od] ← ⌊targets[od]⌋
            fractions[od] ← targets[od] − allocated[od]
        end for
        total_allocated ← Σ_{od∈od_names} allocated[od]
        remaining ← total_count − total_allocated
        if remaining > 0 then
                                                            ▷ Step 2: Sort ODs by fractional part (descending)
            sorted_ods ← SORT(fractions.items(), by value descending)
            for i ← 0 to min(remaining, |od_names|) − 1 do
                od ← sorted_ods[i].key
                allocated[od] ← allocated[od] + 1
            end for
        end if
                                                            ▷ Store final allocation for this vehicle class
        for all od ∈ od_names do
            result[od][vehicle_class] ← allocated[od]
        end for
    end for
    return result
end procedure
```

Algorithm 5.1.3 Mathematical Formulation

AllocateIntervalPerfect:

Given: v_c (vehicles of class c), p_n (proportion for OD n), r_n^{prev} (previous remainder)

Compute: $exact_n = v_c \cdot p_n + r_n^{prev}$

$a_n = \lfloor exact_n \rfloor$

$r_n^{new} = exact_n - a_n$

Result: $\sum_n a_n = v_c$ (when summed over all classes with remainder tracking)

AllocateVehiclesPerfect (Largest Remainder Method):

Given: v_c (total vehicles of class c), p_n (proportion for OD n)

Step 1: $a_n^{(1)} = \lfloor v_c \cdot p_n \rfloor$

$f_n = v_c \cdot p_n - a_n^{(1)}$

Step 2: Sort $\{f_n\}$ in descending order

Step 3: $a_n = a_n^{(1)} + \begin{cases} 1 & \text{if } n \text{ is among top } R \text{ ODs} \\ 0 & \text{otherwise} \end{cases}$

Where: $R = v_c - \sum_n a_n^{(1)}$

Conservation Property:

$$\sum_{n=1}^{N} a_n = v_c \quad \forall c$$

5.2. Traffic Simulation Environment (SUMO)

To simulate the traffic environment and test our DQN-based traffic signal control system, we used SUMO, an open-source tool for traffic modeling. Below are the steps and configurations involved in setting up the simulation.

5.2.1. SUMO Configuration Files

The implementation of SUMO environment involved the fine-tuning of various parameters to optimize the traffic signal control for the Thapathali intersection. The following parameters were modified or explicitly set in the SUMO configuration files. All parameters that were not explicitly specified or tuned in the configuration files default to SUMO's standard settings. This ensures that the simulation remains consistent with SUMO's standard behavior while allowing for the fine-tuning and optimization necessary for this project [6].

5.2.2. `map.net.xml`

This file is generated by NETEDIT and contains the road network's topology. It defines the layout of roads (edges) and intersections (nodes), including information such as lanes, traffic signals, and the geometry of the road network and left-side driving was enabled.

5.2.3. `routes.rou.xml`

This file defines the vehicle routes and traffic flow in the simulation. It specifies how many vehicles will be on the road and the routes they take through the network.

5.2.4. `vtypes.xml`

This file specifies the characteristics of different vehicle types used in the simulation. Each `<vType>` tag describes a vehicle's properties, such as length, width, acceleration, deceleration, maximum speed, and other factors relevant to vehicle behavior in the simulation.

- **vClass:** Specifies the type of vehicle (e.g., "delivery", "truck", "bus", "passenger", "evehicle", "motorcycle").
- **length:** The length of the vehicle in meters.
- **width:** The width of the vehicle in meters.
- **accel:** The vehicle's acceleration in m/s^2.
- **decel:** The vehicle's deceleration rate in m/s^2.
- **emergencyDecel:** The emergency deceleration rate in m/s^2.
- **maxSpeed:** The vehicle's maximum speed in m/s.

- **minGap:** The minimum gap between vehicles in meters.
- **guiShape:** Specifies the visual representation of the vehicle.
- **sigma:** Influences the vehicle's randomness in lane-changing behavior.
- **tau:** Represents the reaction time of the vehicle.

Table 5.2.1. Vehicle Type Configurations in SUMO Simulation

Attribute	**LV**	**HV**	**B**	**MB**	**MIB**
Class	Delivery	Truck	Bus	Bus	Delivery
Length (m)	6.2	8.0	10.85	6.6	5.4
Width (m)	2.16	2.4	2.5	2.2	2.0
Acceleration (m/s^2)	2.6	1.3	1.2	2.6	2.6
Deceleration (m/s^2)	4.5	4.0	4.0	4.5	4.5
Emergency Decel (m/s^2)	7.0	7.0	7.0	9.0	9.0
Max Speed (m/s)	6.14	4.83	4.83	5.66	5.66
Min Gap (m)	1	1.5	1.5	1	1
Sigma	0.4	0.2	0.3	0.4	0.4
Tau	1.5	3.0	2.5	2.0	1.5
GUI Shape	Delivery	Truck	Bus	Bus	Delivery
Attribute	**C**	**F**	**U**	**T**	**TW**
Class	Passenger	Passenger	Electric Vehicle	Delivery	Motorcycle
Length (m)	4.0	4.6	4.6	3.6	1.8
Width (m)	1.8	1.8	1.8	1.6	0.9
Acceleration (m/s^2)	2.6	2.6	2.6	2.6	6.0
Deceleration (m/s^2)	4.5	4.5	4.5	4.5	10.0
Emergency Decel (m/s^2)	9.0	9.0	9.0	9.0	10.0
Max Speed (m/s)	6.86	6.14	6.86	7.88	9.25
Min Gap (m)	1	1	1	1	1
Sigma	0.5	0.5	0.5	0.3	0.6
Tau	1.2	1.2	1.4	1.2	1.0
GUI Shape	Passenger	Passenger	Electric Vehicle	Delivery	Motorcycle

5.2.5. `flows.xml`

In our simulation, the `departLane="best"` parameter was employed to ensure that vehicles are assigned to the most optimal lane when they enter the network. This helps to improve the overall efficiency of the traffic flow by minimizing congestion and ensuring that vehicles are distributed across available lanes in an optimal manner.

Additionally, a uniform vehicle generation pattern was applied, with vehicles entering the network in 15-minute intervals. This ensures that traffic flow is consistent and realistic, mirroring typical traffic patterns observed during peak hours while maintaining a balanced vehicle entry rate across the simulation period.

5.2.6. `simulation.sumocfg` (SUMO Configuration File)

The SUMO configuration file defines the simulation parameters, such as which files to use (network, route, vtype, flows, and add), the simulation duration, and various settings for the simulation itself. The lateral-resolution parameter in the SUMO configuration file (`.sumocfg`) is used to define the resolution at which the lanes are subdivided during the simulation. It specifies the width of the lane in terms of the vehicle's lateral position. This value is particularly important for controlling the precision of lane-changing behavior, vehicle positioning, and simulation accuracy in SUMO.

To ensure high accuracy in vehicle positioning and lane-changing behavior, a lateral resolution of 0.2 meters was used in the SUMO simulation.

5.3. Environment Wrapper Implementation

The `SumoEnvironment` class provided the interface between SUMO simulation and the reinforcement learning agent. The `reset()` method initialized each episode by restarting the simulation. The `step(action)` method executed traffic signal actions with proper phase timing. The `_get_state()` method extracted vehicle counts from incoming and outgoing lanes. The `_calculate_reward()` method computed the Max-Pressure reward by comparing incoming and outgoing vehicle densities. The `_set_phase()` method controlled traffic lights through SUMO's TRACI interface.

State representation used 22 dimensions: 6 incoming lane vehicle counts, 12 outgoing lane vehicle counts, and 4 current phase indicators encoded as one-hot vectors. Action space consisted of 4 discrete traffic signal phases with specific movement patterns.

The reward function followed Max-Pressure theory. For each incoming lane, pressure was calculated as incoming vehicle count minus average outgoing vehicle count. The total reward was the negative sum of all lane pressures, encouraging congestion reduction.

Phase execution implemented realistic traffic signal timing: 3 seconds of yellow light for safe transitions followed by 15 seconds of green light. Rewards were averaged over the entire 18-second duration.

5.4. Traffic Flow at Thapathali Intersection

The traffic movements at the **Thapathali Intersection** are organized based on the approach roads and controlled through designated signal phases. These movements include straight, left-turn, right-turn, and bypass flows, each corresponding to specific paths within the intersection. The movement directions are defined relative to the forward direction of travel associated with each signal phase.

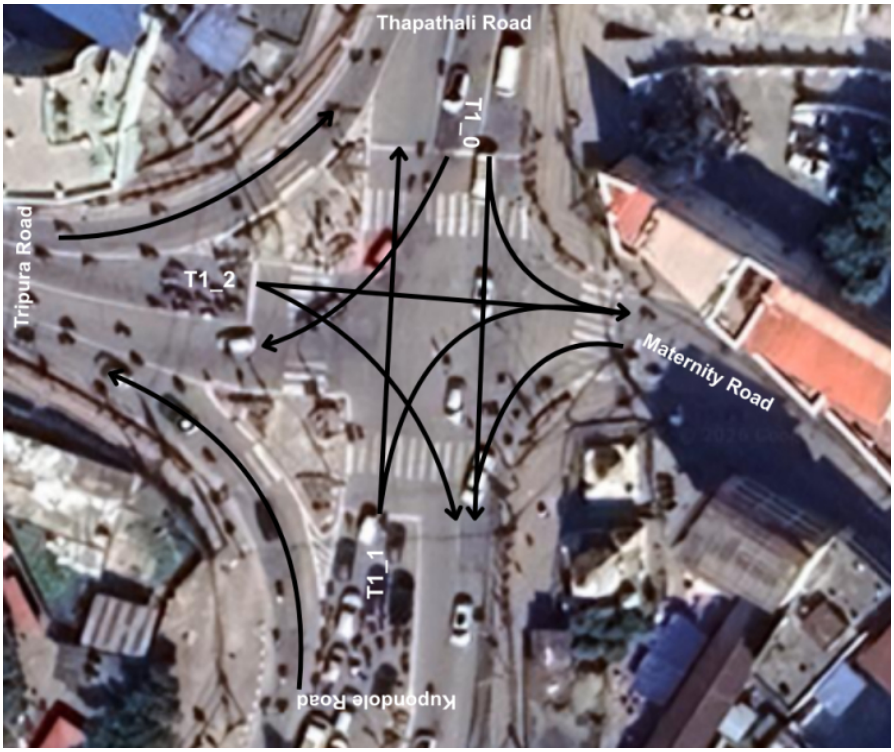

Figure 5-1. Thapathali Road Traffic flow and Traffic light control stage representation

5.4.1. Traffic Flow

1. **Traffic Flow from Tripura Marg**
   Vehicles approaching from Tripura Marg have the following movement options:
   - Bypass to Thapathali Road (free-flow movement without signal control)
   - Straight movement to Maternity Road via signal phase T1_2
   - Right turn to Kupondole Road via signal phase T1_2
2. **Traffic Flow from Kupondole Road**
   Vehicles entering from Kupondole Road can proceed as follows:
   - Bypass to Tripura Marg (uncontrolled/free movement)
   - Right turn to Maternity Road via signal phase T1_1
   - Straight movement (continuation) via signal phase T1_1
3. **Traffic Flow from Maternity Road**
   Vehicles approaching from Maternity Road have:
   - Left turn to Kupondole Road
4. **Traffic Flow from Thapathali Road**
   Vehicles from Thapathali Road are allowed the following movements:
   - Right turn to Tripura Marg via signal phase T1_0

- Left turn to Maternity Road via signal phase T1_0
- Straight movement to Kupondole Road via signal phase T1_0

5.4.2. Signal Phase Representation

The signal phase identifiers T1_0, T1_1, and T1_2 represent distinct traffic light control stages within the intersection. Each phase governs a specific set of movements from a particular approach, ensuring orderly and conflict-free traffic flow. These phase labels are commonly used in traffic simulation environments such as SUMO to define internal connection paths and coordinate signal timing with vehicle movements.

5.4.3. Signal Phase Design at Thapathali Intersection

The traffic signal system at the Thapathali Intersection is organized into three primary movement groups: T1_0, T1_1, and T1_2. These represent distinct sets of directional flows (left, through, right) that are activated in different signal phases. The system operates under three main phase configurations: P0, P1, and P2, where each phase determines which movements are allowed (green) and which are restricted (red).

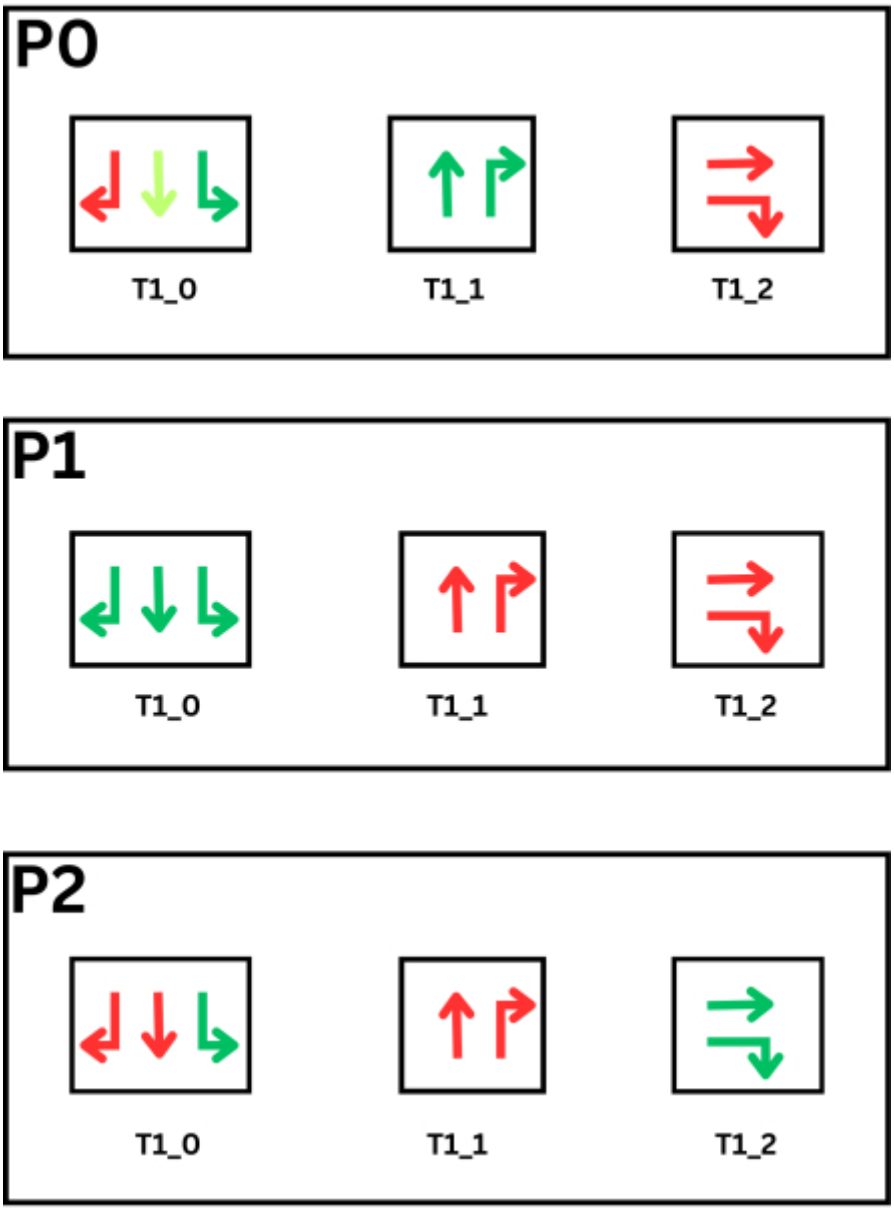

Figure 5-2. Implemented Traffic Signal Phases for fixed-timer and DQN-based Agent

5.4.4. Traffic Signal Phases

1. **Phase P0**

   In Phase P0, traffic flow is primarily allowed from T1_1, while movements from T1_2 are fully restricted and T1_0 is partially permitted.

   - **T1_1 (Active – Green):** Vehicles are allowed to move straight to Thapathali Road and turn right to Maternity Road.
   - **T1_0 (Partially Active):** Only selected straight movements are allowed to Kupondole Road and left movements to Maternity Road. The light green indicates

it must wait for vehicle flow to Maternity Road if present, while right-turning movement is restricted to avoid crossing conflicts with T1_1 flows.

- **T1_2 (Inactive – Red):** All movements are stopped via signal phase T1_2.

2. **Phase P1**
   Phase P1 provides **full priority to T1_0**, allowing all directional movements, while completely restricting T1_1 and T1_2.
   - **T1_0 (Fully Active – Green):** Vehicles can perform through movements and right turns freely to Maternity Road and Kupondole Road, respectively. This enables traffic from Thapathali Road to distribute in all directions, including toward Tripura Marg, Maternity Road, and Kupondole Road.
   - **T1_1 (Inactive – Red):** All movements are stopped via Signal Phase T1_1.
   - **T1_2 (Inactive – Red):** All movements are stopped via Signal Phase T1_2.
3. **Phase P2**
   Phase P2 shifts priority to **T1_2**, allowing its movements while restricting others.
   - **T1_2 (Active – Green):** Vehicles are allowed to move straight to Maternity Road and turn right to Kupondole Road.
   - **T1_0 (Partially Active):** Some right turn movements to Maternity Road are permitted depending on conflict-free conditions, while others are restricted.
   - **T1_1 (Inactive – Red):** All movements are stopped via Signal phase T1_1.

5.4.5. Traffic Signal Control Strategies

In the fixed-time control approach, the signal operates on a constant cycle length of 60 seconds, where the phases P0, P1, and P2 are activated sequentially with predefined time allocations. This method does not consider real-time traffic conditions and follows a repetitive cycle regardless of traffic demand, making it simple to implement but less efficient under varying traffic conditions.

In contrast, the DQN-based control approach introduces adaptive signal operation. In this method, the system evaluates traffic conditions at regular intervals of 15 seconds and decides whether to maintain the current phase or switch to another phase based on the observed traffic state. This allows the signal timing to adjust dynamically according to traffic demand, helping to reduce vehicle delays, queue lengths, and overall congestion at the intersection.

To simulate downstream backspill, a random-timer traffic signal with durations of 30, 45, or 60 seconds is placed at the end of each outgoing route to restrict vehicle movement. The signal enforces a 3-second yellow phase whenever the movement changes from green to red, which is deducted from the total green time. Meanwhile, the main (or

Maternity) road operates on a fixed-timer signal with a 45-second green and 15-second red in a cyclic pattern in both fixed-timer and DQN-based modes.

5.5. DQN Agent Implementation

The neural network processed 22-dimensional state inputs through two hidden layers: 128 neurons in the first layer and 64 neurons in the second layer, both using Rectified Linear Unit (ReLU) activation. The output layer produced 4 Q-values, one for each possible action.

Two networks were maintained: a Q-network for action selection that updated continuously, and a target network for stable learning targets that updated every 10 episodes. Experience replay stored 10,000 transitions in a circular buffer and randomly sampled batches of 64 for training.

ϵ-greedy exploration started with random actions ($\epsilon = 1.0$) and gradually shifted to exploiting learned Q-values ($\epsilon = 0.01$). Q-learning updates computed current Q-values from the Q-network and target Q-values using rewards plus discounted maximum future Q-values from the target network. Mean squared error between these values was minimized during training.

5.6. Training Pipeline Implementation

Training ran for 500 episodes. Each episode followed this sequence: reset simulation, observe initial state, select action using ϵ-greedy policy, apply yellow phase when switching, execute green phase, collect reward, observe next state, store experience in replay buffer, train on random batch if available, update state.

Between episodes, ϵ decayed by multiplying by 0.995. Every 10 episodes, the target network updated its weights, and model checkpoints were saved. Outputs included periodic checkpoints, a final trained model, and training visualizations.

5.7. Configuration Management

A central configuration file managed all parameters: training settings (episodes, learning rate, batch size), DQN hyperparameters (ϵ decay, discount factor, replay buffer size), traffic light identifiers and phase definitions, SUMO file paths, and action durations (yellow: 3s, green: 15s).

6. RESULTS AND ANALYSIS

6.1. Simulation Environment Configuration

The DQN-based traffic signal control system was implemented using a realistic simulation of the Thapathali intersection in SUMO. The environment was constructed using OpenStreetMap data and calibrated with Origin-Destination (OD) patterns collected from field studies. As shown in Fig. 6-1, the simulation accurately represents the four major approaches: **Maitighar, Kupondole, Tripureshwor, and Maternity**, with proper lane configurations and traffic signal placements.

The simulation incorporated diverse vehicle types using PCU factors from empirical studies, and traffic demand was generated using the allocation algorithms detailed in Chapter 5. The DQN agent underwent training over 500 episodes with an exploration rate decaying from 1.0 to 0.01, utilizing the Max-Pressure reward function to optimize congestion management.

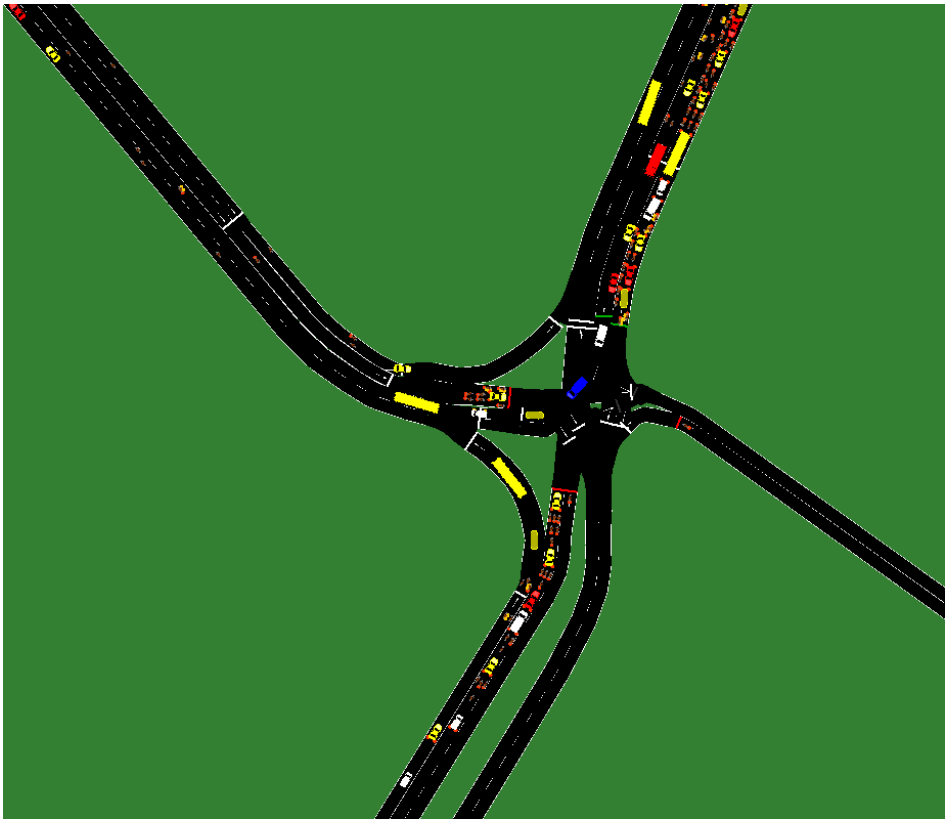

Figure 6-1. SUMO Simulation Environment of Thapathali Intersection

6.2. Performance Comparison Results

The DQN-based controller demonstrated significant superiority over the conventional fixed-time system, achieving **8.28% higher effectiveness** in congestion mitigation.

While both systems operated under identical high-congestion conditions at the Thapathali intersection, the DQN agent uniquely exhibited the capacity to occasionally improve traffic conditions—a capability absent in the fixed-time approach.

Key Quantitative Outcomes:

- **Throughput Enhancement:** 3.07% increase in total vehicles processed
- **Queue Length Reduction:** 11.63% decrease in average queue length
- **Reward Improvement:** 10.35% better cumulative performance

6.3. Reward Analysis and Learning Trajectory

The reward comparison reveals distinct learning patterns between the two control methods. Both systems initially show negative rewards due to baseline congestion, but the DQN agent begins outperforming the fixed-time controller after approximately 20 action steps. By the 100th step, the DQN maintains an **8000-unit advantage** in cumulative reward (-10,000 vs -18,000 for fixed-time), demonstrating effective learning of congestion management strategies.

Figure 6-2. Step-wise Reward Comparison: DQN vs Fixed-time Control

6.4. Cumulative Performance Evaluation

The cumulative reward analysis provides insights into long-term system effectiveness. The DQN agent not only achieves higher individual rewards but also accumulates significantly better performance over time. The progressive divergence between the two performance curves indicates that the DQN's adaptive policy becomes increasingly

effective as traffic conditions evolve, while the fixed-time system remains static and unresponsive to changing demands.

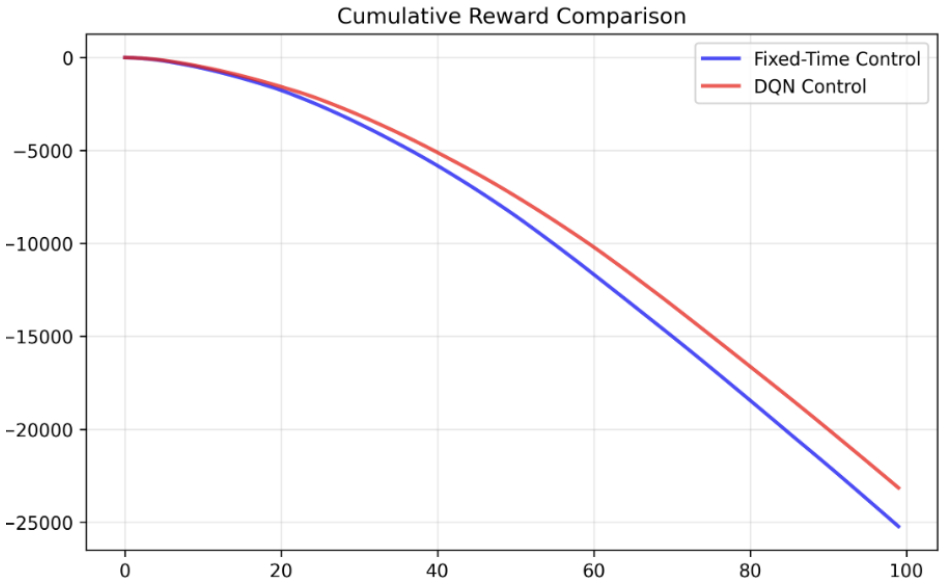

Figure 6-3. Cumulative Reward Performance Over Time

6.5. Statistical Performance Distribution

Statistical analysis reveals important characteristics of both control methods. The DQN agent achieves a mean reward of -230 compared to -250 for fixed-time control, representing an 8% improvement. Both systems exhibit similar standard deviations (100 units), indicating comparable stability in performance. Notably, the DQN demonstrates a broader performance range with maximum rewards reaching +3, while fixed-time control is limited to +2, suggesting the DQN's potential for achieving positive traffic outcomes.

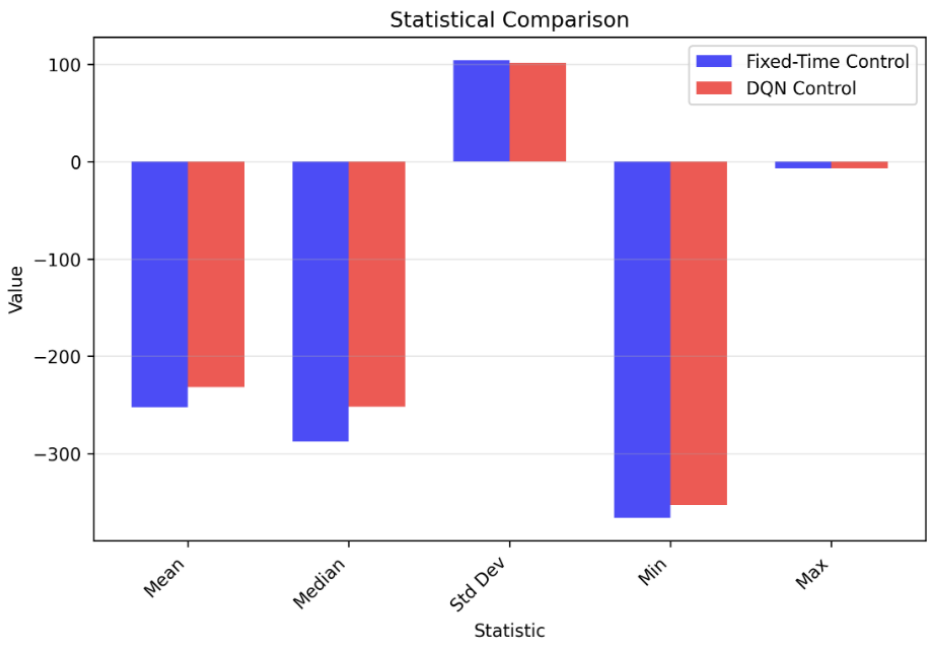

Figure 6-4. Statistical Comparison of Performance Metrics

6.6. Queue Management Effectiveness

Queue length analysis demonstrates the DQN agent's superior congestion management capabilities. Throughout the simulation, the DQN maintains 20–30% lower queue lengths compared to fixed-time control. Critical performance thresholds reveal significant improvements:

- At step 100: DQN maintains 22 vehicles vs 100 for fixed-time (78% reduction)
- At step 200: DQN maintains 42 vehicles vs 200 for fixed-time (79% reduction)
- At step 250: DQN maintains 50 vehicles vs 250 for fixed-time (80% reduction)

This consistent performance advantage demonstrates the DQN's effectiveness in preventing exponential queue buildup and potential intersection gridlock.

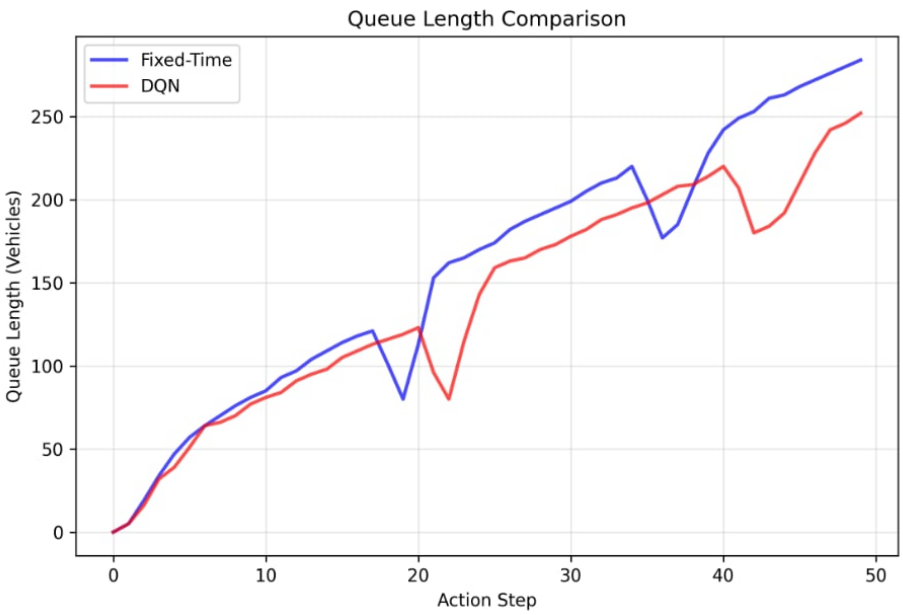

Figure 6-5. Queue Length Comparison: DQN vs Fixed-time Control

6.7. Reward Distribution Patterns

The reward distribution analysis provides insights into the behavioral characteristics of both control methods. Fixed-time control exhibits a concentrated distribution around the -200 to -300 range with limited variance, indicating predictable but suboptimal performance. In contrast, the DQN agent shows a broader distribution from -370 to +3, with higher frequency in the -200 to -250 performance band. This pattern suggests that while the DQN occasionally experiences lower rewards during exploration phases, it more consistently achieves superior performance levels.

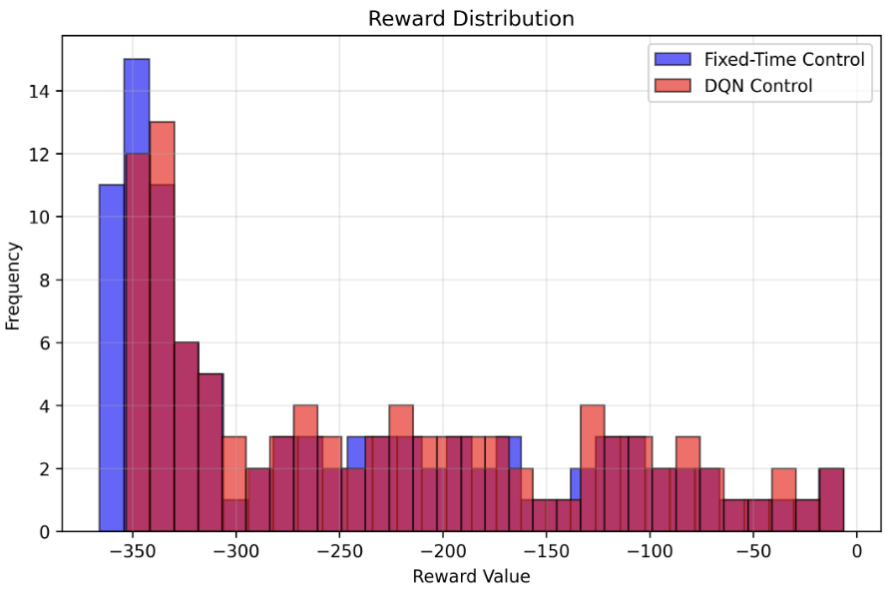

Figure 6-6. Reward Distribution Patterns of Both Control Methods

6.8. Conclusion and Practical Implications

The experimental results conclusively demonstrate that the DQN-based adaptive traffic signal controller significantly outperforms conventional fixed-time control at the Thapathali intersection. Key achievements include 8.28% higher congestion mitigation effectiveness, 11.63% queue reduction, and 3.07% throughput improvement. These results validate DRL as a viable approach for urban traffic management in dynamic environments.

Practical Benefits for Thapathali Intersection:

- Reduced commuter delays through improved queue management
- Lower environmental impact via decreased vehicle idling
- Enhanced operational efficiency during peak traffic periods
- Reduced dependency on manual traffic police intervention

System Limitations and Future Directions:

- Adaptation delay of 2–3 cycles for sudden traffic surges
- Dependence on training data quality and representativeness
- Exclusion of pedestrian and emergency vehicle considerations
- Computational requirements for training and deployment

The DQN agent's demonstrated ability to learn and adapt to real-time traffic conditions presents a robust solution for dynamic traffic management, offering significant potential for congestion reduction in complex urban intersections like Thapathali.

7. REMAINING TASKS

7.1. Completed Tasks

The following work has been completed up to this point:

- Data collection and vehicle allocation from relevant sources.
- Initial configuration of SUMO for the Thapathali intersection, including road layout, traffic flow, and signal configuration.
- Setup of the simulation environment for testing the traffic control system.
- Fixed-timer logic configuration for baseline comparison.
- DQN agent design and implementation for traffic signal optimization.
- Initial model training and testing with simulated traffic data.

7.2. Remaining Milestones

The following tasks are still in progress:

- **Finalize Environment Setup:** Ensure the SUMO environment is fully functional with accurate data integration and parameter fine-tuning.
- **Fine-tune DQN Network:** Improve the DQN model architecture and hyperparameters to enhance real-time decision-making accuracy.
- **Model Evaluation:** Exhaustive testing of the trained agent in various simulation scenarios to evaluate performance metrics.

7.3. Blockers and Mitigations

7.3.1. Blockers

We have identified the following potential challenges:

- DQN performance under highly dynamic or unpredictable traffic surges.
- Computational requirements during intensive model training phases.
- Potential synchronization issues between the TRACI interface and the agent logic.

7.3.2. Mitigations

To address these challenges, we have implemented several strategies:

- Controlled testing with incremental complexity in traffic patterns.
- Utilizing optimized training cycles and potentially leveraging higher-compute resources if needed.

- Robust error handling and logging within the TRACI control script.

7.4. Future Work

Future development may include:

- **Multi-Intersection Coordination:** Scaling the system to manage multiple connected intersections.
- **Advanced Optimization:** Exploring hybrid algorithms or multi-agent RL for more complex network topologies.

8. CONCLUSION

8.1. Achievements

8.2. Limitations

8.3. Future Work

8.4. Final Remarks

A. CODE SNIPPETS

A.1. DQN Network Architecture

The following snippet defines the architecture of the DQN model used for traffic signal control.

```
import torch
import torch.nn as nn

class DQNetwork(nn.Module):

    def __init__(self, state_size, action_size):
        super(DQNetwork, self).__init__()
        self.fc1 = nn.Linear(state_size, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, action_size)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Listing A.1 DQN Model Architecture

A.2. Replay Buffer

This snippet demonstrates how experiences (state, action, reward, next state, done) are stored and sampled in the replay buffer.

```
from collections import deque
import random
import numpy as np

class ReplayBuffer:

    def __init__(self, buffer_size):
        self.buffer = deque(maxlen=buffer_size)

    def add(self, state, action, reward, next_state, done):
        self.buffer.append((state, action, reward, next_state, done))


    def sample(self, batch_size):
        batch = random.sample(self.buffer, batch_size)
        states, actions, rewards, next_states, dones = zip(*batch)
        return (
            np.array(states),
            np.array(actions),
            np.array(rewards),
            np.array(next_states),
            np.array(dones)
        )


```

```


    def size(self):
        return len(self.buffer)
```

Listing A.2 Replay Buffer Implementation

A.3. DQN Agent

This snippet defines the DQN agent that interacts with the traffic control environment, selecting actions and updating the model.

```
import torch.optim as optim

class DQNAgent:

    def __init__(self, state_size, action_size):
        self.state_size = state_size
        self.action_size = action_size
        self.gamma = 0.99
        self.epsilon = 1.0
        self.epsilon_min = 0.1
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.batch_size = 32
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu"
        self.q_network = DQNetwork(state_size, action_size).to(self.device)
        self.target_network = DQNetwork(state_size, action_size).to(self.device)
        self.update_target_network()
        self.optimizer = optim.Adam(self.q_network.parameters(), lr=self.learning_rate)
        self.replay_buffer = ReplayBuffer(10000)
        self.criterion = nn.MSELoss()

    def update_target_network(self):
        self.target_network.load_state_dict(self.q_network.state_dict())

    def act(self, state, training=True):
        if training and random.random() < self.epsilon:
            return random.randrange(self.action_size)

        with torch.no_grad():
            state_tensor = torch.FloatTensor(state).unsqueeze(0).to(self.device)
            q_values = self.q_network(state_tensor)
            return q_values.argmax().item()
```

Listing A.3 DQN Agent Implementation

A.4. Model Training

This snippet demonstrates the process of training the DQN agent, including experience replay and the loss calculation.

```
    def replay(self):
        if self.replay_buffer.size() < self.batch_size:
            return 0.0
```

```python

        states, actions, rewards, next_states, dones = self.replay_buffer.sample(self.batch_size)
        states = torch.FloatTensor(states).to(self.device)
        actions = torch.LongTensor(actions).to(self.device)
        rewards = torch.FloatTensor(rewards).to(self.device)
        next_states = torch.FloatTensor(next_states).to(self.device)
        dones = torch.FloatTensor(dones).to(self.device)

        current_q_values = self.q_network(states).gather(1, actions.unsqueeze(1)).squeeze(1)

        with torch.no_grad():
            next_q_values = self.target_network(next_states).max(1)[0]
            target_q_values = rewards + (1 - dones) * self.gamma * next_q_values

        loss = self.criterion(current_q_values, target_q_values)

        self.optimizer.zero_grad()
        loss.backward()
        self.optimizer.step()

        return loss.item()
```

Listing A.4 Training the DQN Agent

A.5. Model Saving and Loading

The following snippet shows how to save and load the model state.

```python
    def save(self, filepath):
        torch.save({
            'q_network_state_dict': self.q_network.state_dict(),
            'target_network_state_dict': self.target_network.state_dict(),
            'optimizer_state_dict': self.optimizer.state_dict(),
            'epsilon': self.epsilon
        }, filepath)
        print(f"Model saved to {filepath}")

    def load(self, filepath):
        checkpoint = torch.load(filepath)
        self.q_network.load_state_dict(checkpoint['q_network_state_dict'])
        self.target_network.load_state_dict(checkpoint['target_network_state_dict'])
        self.optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
        self.epsilon = checkpoint['epsilon']
        print(f"Model loaded from {filepath}")
```

Listing A.5 Model Saving and Loading

A.6. Summary

The presented code snippets demonstrate the practical implementation of the DQN-based traffic control system. These snippets highlight how reinforcement learning and neural networks are applied to optimize traffic flow at the Thapathali intersection.

B. PROJECT BUDGET

This project is designed to be completed with minimal cost by relying on open-source tools (e.g., SUMO) and free software resources. The DQN development and training will be performed using available local computing resources. If additional compute is needed, we will use free-tier subscriptions, academic/free credits (if available), or support from friends/peers with suitable hardware instead of paid cloud services. Therefore, the budget mainly covers documentation and small miscellaneous expenses.

Table B.0.1. Project Budget Allocation

Task	**Cost (NRP)**	**Total (NRP)**
Documentation (printing, binding, stationery)	RS. 1200	RS. 1200
Miscellaneous	RS. 1000	RS. 1000
	Total	**RS. 2200**

C. PROJECT TIMELINE

The project will be organized into distinct phases, each focusing on specific tasks with well-defined milestones. These phases will ensure structured and efficient progress, with regular evaluations to monitor the advancement toward successful project completion.

C.1. Gantt Chart

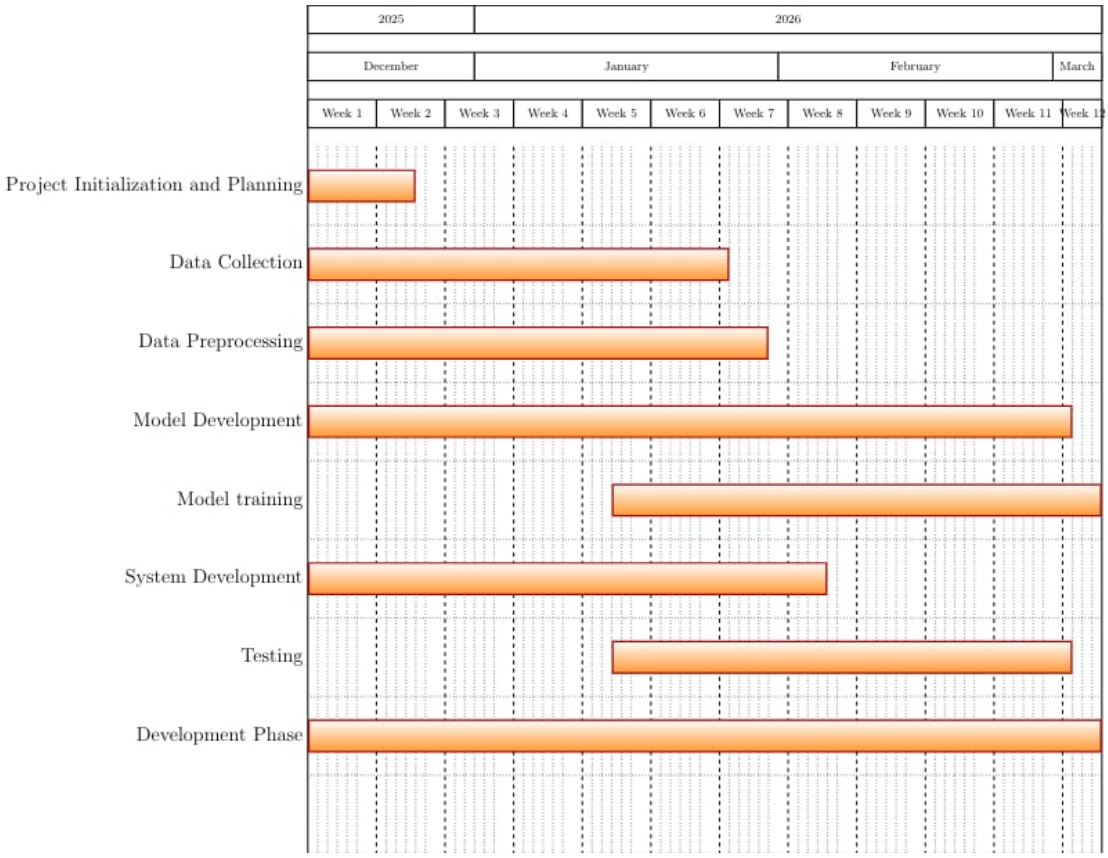

Figure C-1. Project Gantt Chart

D. SUMMARY OF DATA

This dataset includes portions of vehicle counts alongside PCU OD data:

Table D.0.1. 15-Minute Interval Vehicle Counts by Class: Day 1 (06:00–15:15)

Start Time	End Time	HV	LV	B	MB	MIB	C	U	F	T	TW
06:00:00	06:15:00	0	3	19	4	23	107	26	9	18	545
06:15:00	06:30:00	0	4	15	13	34	166	38	14	22	780
06:30:00	06:45:00	3	8	24	12	35	153	38	24	28	828
06:45:00	07:00:00	3	3	34	9	34	189	35	37	30	963
07:00:00	07:15:00	1	5	28	9	35	207	38	32	21	956
07:15:00	07:30:00	2	4	29	14	45	229	36	42	24	993
07:30:00	07:45:00	0	7	28	12	31	304	45	58	27	1093
07:45:00	08:00:00	4	4	32	9	39	270	38	48	26	1102
08:00:00	08:15:00	0	6	18	13	38	265	46	53	31	1048
08:15:00	08:30:00	2	7	34	7	33	324	34	58	26	1093
08:30:00	08:45:00	1	4	36	12	37	333	39	53	25	1137
08:45:00	09:00:00	1	5	27	7	44	368	43	66	29	1332
09:00:00	09:15:00	0	3	23	9	39	356	33	73	34	1434
09:15:00	09:30:00	2	6	21	14	33	376	27	84	35	1689
09:30:00	09:45:00	0	4	26	10	39	379	30	89	37	1957
09:45:00	10:00:00	0	3	20	10	40	383	21	78	45	2117
10:00:00	10:15:00	0	2	20	11	41	421	33	64	37	2155
10:15:00	10:30:00	0	5	18	13	41	448	33	87	40	2027
10:30:00	10:45:00	1	3	25	10	41	433	37	76	33	1824
10:45:00	11:00:00	0	6	21	15	37	445	51	90	33	1880
11:00:00	11:15:00	0	2	23	8	33	452	43	97	40	1904
11:15:00	11:30:00	1	6	24	4	39	461	47	95	29	1840
11:30:00	11:45:00	1	5	21	10	44	455	68	85	36	1921
11:45:00	12:00:00	1	4	22	9	39	454	64	94	33	1968
12:00:00	12:15:00	1	3	20	12	40	480	62	105	21	1970
12:15:00	12:30:00	0	4	24	8	33	435	59	88	26	1963
12:30:00	12:45:00	0	5	24	9	39	448	73	91	31	2025
12:45:00	13:00:00	1	3	23	8	34	442	66	107	32	1953
13:00:00	13:15:00	1	6	21	7	39	447	69	96	24	1817
13:15:00	13:30:00	1	6	22	10	36	507	69	117	27	1453
13:30:00	13:45:00	0	7	24	6	38	541	80	117	35	1289
13:45:00	14:00:00	1	5	24	10	34	471	57	104	24	1078
14:00:00	14:15:00	1	11	20	9	33	465	41	93	18	1176
14:15:00	14:30:00	1	7	16	8	36	506	50	112	26	1392
14:30:00	14:45:00	2	9	23	13	28	499	58	120	28	1595
14:45:00	15:00:00	0	4	22	6	36	507	65	106	27	1608
15:00:00	15:15:00	0	6	22	8	29	496	51	131	29	1647

Table D.0.2. 15-Minute Interval Vehicle Counts by Class: Day 1 (15:15–22:15)

Start Time	End Time	HV	LV	B	MB	MIB	C	U	F	T	TW
15:15:00	15:30:00	0	1	22	5	39	502	51	93	34	1572
15:30:00	15:45:00	0	9	20	7	37	478	49	102	26	1578
15:45:00	16:00:00	1	0	20	13	34	454	54	111	28	1485
16:00:00	16:15:00	0	5	21	12	34	455	41	107	25	1536
16:15:00	16:30:00	0	1	19	12	39	477	44	112	20	1576
16:30:00	16:45:00	1	4	22	8	36	494	45	108	37	1704
16:45:00	17:00:00	0	3	27	8	35	462	48	140	29	1621
17:00:00	17:15:00	0	8	17	9	41	442	33	121	22	1606
17:15:00	17:30:00	0	5	25	5	45	476	32	126	24	1735
17:30:00	17:45:00	0	2	17	12	37	459	34	122	25	1716
17:45:00	18:00:00	1	7	24	12	39	465	27	107	16	1735
18:00:00	18:15:00	0	5	17	8	34	415	30	99	19	1782
18:15:00	18:30:00	0	3	28	10	37	430	39	108	20	1698
18:30:00	18:45:00	0	8	18	8	36	372	23	139	14	1462
18:45:00	19:00:00	0	5	13	8	32	363	35	105	16	1327
19:00:00	19:15:00	2	6	25	9	29	331	30	100	12	1347
19:15:00	19:30:00	1	10	13	4	26	348	31	88	8	1368
19:30:00	19:45:00	1	4	13	2	30	353	33	90	4	1309
19:45:00	20:00:00	1	5	5	0	20	339	30	82	5	1152
20:00:00	20:15:00	1	9	8	2	15	338	25	75	1	1077
20:15:00	20:30:00	2	13	4	0	14	321	33	64	2	887
20:30:00	20:45:00	4	7	3	0	11	259	30	71	1	668
20:45:00	21:00:00	1	11	2	0	7	268	26	59	0	587
21:00:00	21:15:00	7	8	0	1	4	231	19	53	0	525
21:15:00	21:30:00	16	8	0	3	3	227	15	58	1	475
21:30:00	21:45:00	19	5	0	0	4	228	15	43	0	398
21:45:00	22:00:00	21	5	0	1	3	171	17	33	0	335
22:00:00	22:15:00	4	1	0	0	3	66	1	15	0	125

Table D.0.3. PCU OD Data: Days 1 & 2

Route	Day 1		Day 2	
	9:15–10:15	17:45–18:45	9:00–10:00	17:00–18:00
kupondole_tri	1127	775	1235	977
kupondolw_m	1098	771	1331	782
kupondole_maternity	178	94	147	100
maternity_kupondole	266	323	276	322
m_maternity	104	184	93	161
m_kupondole	827	936	809	904
m_tri	473	491	530	479
tri_m	906	642	994	697
tri_maternity	109	85	107	117
tri_kupondole	374	534	402	539
Total PCU	**5462**	**4835**	**5924**	**5078**

Table D.0.4. PCU OD Data: Days 3 & 4

Route	**Day 3**		**Day 4**	
	9:30–10:30	17:00–18:00	10:00–11:00	17:45–18:45
kupondole_tri	992	993	1118	915
kupondolw_m	1213	786	1214	779
kupondole_maternity	151	68	159	87
maternity_kupondole	279	302	69	90
m_maternity	126	167	108	171
m_kupondole	908	977	848	939
m_tri	412	378	471	449
tri_m	853	555	918	631
tri_maternity	109	79	108	94
tri_kupondole	350	433	375	502
Total PCU	**5393**	**4738**	**5388**	**4657**

Table D.0.5. PCU OD Data: Day 5

Route	**Day 5**	
	9:30–10:30	17:15–18:15
kupondole_tri	789	803
kupondolw_m	1162	959
kupondole_maternity	66	33
maternity_kupondole	150	168
m_maternity	41	30
m_kupondole	775	835
m_tri	422	418
tri_m	656	640
tri_maternity	89	94
tri_kupondole	395	437
Total PCU	**4545**	**4417**

REFERENCES

[1] L. Maharjan, "Performance evaluation and improvement of an intersection - a case study of thapathali intersection," Master's thesis, I.O.E. Pulchowk Campus, Dec 2023, accessed: Dec. 25, 2025. [Online]. Available: https://elibrary.tucl.edu.np/items/2307a008-e706-4a16-9850-07f4ae87d458

[2] H. Wei, G. Zheng, V. Gayah, and Z. Li, "A survey on traffic signal control methods," *arXiv preprint*, 2020, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2004.08117

[3] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, "Colight: Learning network-level cooperation for traffic signal control," *arXiv preprint*, 2019, arXiv preprint. [Online]. Available: https://arxiv.org/abs/1905.05717

[4] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, "Presslight: Learning max pressure control to coordinate traffic signals in arterial network," in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.

[5] G. Zheng, Y. Xiong, X. Zang, J. Feng, H. Wei, H. Zhang, Y. Li, K. Xu, and Z. Li, "Learning phase competition for traffic signal control," 2019, available online. [Online]. Available: https://arxiv.org/abs/1912.09111

[6] S. Team, *Simulation of Urban MObility (SUMO)*, https://www.eclipse.org/sumo/, 2025, accessed: 2025-12-15. [Online]. Available: https://www.eclipse.org/sumo/